

STUDENT 25: CONSTRUCTION COST & RESOURCES

A1 :Fragment & Recombine Main Fact (≤10 rows)

The screenshot displays a PostgreSQL IDE interface with two query windows. The top window shows a query to create schemas, and the bottom window shows a query to create a table with specific constraints. Both queries were executed successfully.

Query 1:

```
--1. Create schemas (simulate Node_A and Node_B)
CREATE SCHEMA IF NOT EXISTS node_a;
CREATE SCHEMA IF NOT EXISTS node_b;
```

Data Output:

```
CREATE SCHEMA

Query returned successfully in 116 msec.
```

Query 2:

```
--2. Create fragment tables with RANGE checks (deterministic rule)
-- Node A fragment (expense_id 1..5)
CREATE TABLE IF NOT EXISTS node_a.expense_a (
  expense_id    INTEGER PRIMARY KEY,
  project_id    INTEGER NOT NULL,
  contractor_id INTEGER,
  amount        NUMERIC(12,2) NOT NULL CHECK (amount >= 0),
  expense_date  DATE NOT NULL,
  description   TEXT,
  CHECK (expense_id BETWEEN 1 AND 5)
);
```

Data Output:

```
CREATE TABLE

Query returned successfully in 93 msec.
```


QueryQuery History

Scratch Pad X

1

--4. Create expense_all view on Node_A that unions local + remote fragment

2

CREATE OR REPLACE VIEW node_a.expense_all AS

3

SELECT * FROM node_a.expense_a

4

UNION ALL

5

SELECT * FROM node_b.expense_b;

6

Data OutputMessagesNotifications

CREATE VIEW

Query returned successfully in 78 msec.

QueryQuery History

Scratch Pad X

1

--5. Validation queries – COUNT(*) and checksum

2

-- Per-fragment counts & checksum

3

SELECT 'node_a.expense_a' AS source, COUNT(*) AS cnt, SUM(MOD(expense_id,9

4

FROM node_a.expense_a

5

UNION ALL

6

SELECT 'node_b.expense_b', COUNT(*), SUM(MOD(expense_id,97))

7

FROM node_b.expense_b

8

UNION ALL

9

SELECT 'node_a.expense_all', COUNT(*), SUM(MOD(expense_id,97))

10

FROM node_a.expense_all;

Data OutputMessagesNotifications

Showing rows: 1 to 3Page No: 1 of 1

	source text	cnt bigint	checksum bigint
1	node_a.expense_a	5	15
2	node_b.expense_b	5	40
3	node_a.expense_all	10	55

Query

Query History

Scratch Pad

1

-- Show node_a rows

2

SELECT * FROM node_a.expense_a ORDER BY expense_id;

3

-- Show node_b rows

4

SELECT * FROM node_b.expense_b ORDER BY expense_id;

5

-- Show the combined view (first 10 rows)

6

SELECT * FROM node_a.expense_all ORDER BY expense_id;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 10

Page No: 1

of 1

	expense_id integer	project_id integer	contractor_id integer	amount numeric (12,2)	expense_date date	description text
1	1	101	1001	120.00	2025-10-01	Office supplies
2	2	101	1002	500.00	2025-10-02	Site materials
3	3	102	1001	250.00	2025-10-03	Transport
4	4	103	1003	600.00	2025-10-04	Consulting
5	5	103	1004	75.50	2025-10-05	Misc
6	6	201	2001	80.00	2025-10-01	Fuel
7	7	201	2002	300.00	2025-10-02	Equipment rental
8	8	202	2001	430.00	2025-10-03	Subcontractor
9	9	203	2003	150.00	2025-10-04	Safety gear
10	10	203	2004	220.00	2025-10-05	Catering

A2 :Database Link & Cross-Node Join (3–10 rows result)

Query	Query History	Scratch Pad
<pre> 1 --1. Enable FDW extension 2 CREATE EXTENSION IF NOT EXISTS postgres_fdw; 3 </pre>		
Data Output	Messages	Notifications
CREATE EXTENSION		
Query returned successfully in 222 msec.		

Query

Query History

Scratch Pad X

1

--2. Create a "database link" from Node A → Node B (simulated connection)

2

CREATE SERVER proj_link

3

FOREIGN DATA WRAPPER postgres_fdw

4

OPTIONS (host 'localhost', dbname 'CCR', port '5432');

5

Data Output

Messages

Notifications

CREATE SERVER

Query returned successfully in 99 msec.

Query

Query History

Scratch Pad X

1

--3. Create user mapping (required by FDW)

2

CREATE USER MAPPING FOR CURRENT_USER

3

SERVER proj_link

4

OPTIONS (user 'postgres', password '1234');

5

Data Output

Messages

Notifications

CREATE USER MAPPING

Query returned successfully in 124 msec.

Query

Query History

Scratch Pad X

1

--4. Expose a remote table on Node B to Node A

2

--a. Create table on Node B:

3

CREATE TABLE IF NOT EXISTS node_b.contractor (

4

contractor_id INTEGER PRIMARY KEY,

5

contractor_name VARCHAR(50)

6

);

7

8

INSERT INTO node_b.contractor VALUES

9

(2001, 'BuildCo Ltd'),

10

(2002, 'SteelWorks'),

11

(2003, 'QuickFix Services'),

12

(2004, 'ProConstruct');

13

Data Output

Messages

Notifications

INSERT 0 4

Query returned successfully in 148 msec.

Query

Query History

Scratch Pad X

1

--b. Expose Contractor from Node B to Node A:

2

IMPORT FOREIGN SCHEMA node_b

3

LIMIT TO (contractor)

4

FROM SERVER proj_link

5

INTO node_a;

6

Data Output

Messages

Notifications

IMPORT FOREIGN SCHEMA

Query returned successfully in 198 msec.

Folder

Save

Undo

Filter

No limit

Stop

Play

Refresh

Export

Import

Help

Search

Query

Query History

Scratch Pad X

1

--5. Test remote SELECT

2

SELECT * FROM node_a.contractor LIMIT 5;

3

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 4

Page No: 1 of 1

	contractor_id integer	contractor_name character varying (50)
1	2001	BuildCo Ltd
2	2002	SteelWorks
3	2003	QuickFix Services
4	2004	ProConstruct

Total rows: 4Query complete 00:00:00.145CRLF Ln 3, Col 1

Query

Query History

Scratch Pad X

```

1  -- 2 - Parallel Aggregation
2  -- Enable parallelism
3  SET max_parallel_workers_per_gather = 8;
4
5  EXPLAIN (ANALYZE, BUFFERS)
6  SELECT category, SUM(amount) AS total_amount
7  FROM expense_all
8  GROUP BY category;
9

```

Data Output

Messages

Notifications

Showing rows: 1 to 14

Page No: 1

of 1

QUERY PLAN

text

1

HashAggregate (cost=37.60..40.10 rows=200 width=150) (actual time=0.128..0.136 rows=5 loops=1)

2

Group Key: expense_a.category

3

Batches: 1 Memory Usage: 40kB

4

Buffers: shared hit=2

5

-> Append (cost=0.00..33.20 rows=880 width=150) (actual time=0.053..0.082 rows=10 loops=1)

6

Buffers: shared hit=2

7

-> Seq Scan on expense_a (cost=0.00..14.40 rows=440 width=150) (actual time=0.051..0.054 rows=5 l...

8

Buffers: shared hit=1

9

-> Seq Scan on expense_b (cost=0.00..14.40 rows=440 width=150) (actual time=0.020..0.022 rows=5 l...

10

Buffers: shared hit=1

11

Planning:

12

Buffers: shared hit=9

13

Planning Time: 13.045 ms

14

Execution Time: 0.294 ms

Total rows: 14

Query complete 00:00:00.240

CRLF

Ln 9, Col 1

3 — Compare Results

Mode	Workers	Plan Nodes	Total Time	Buffers	Notes
Serial	0	HashAggregate	e.g., 1.25 ms	12	Single-threaded
Parallel	8	Gather + Partial HashAggregate	e.g., 0.75 ms	10	Used parallel workers

A4 :Two-Phase Commit & Recovery (2 rows)

Query

Query History

Scratch Pad X

```

1  --1: Set up sample tables
2  -- Local node (Node_A)
3  CREATE TABLE expense_local (
4      id SERIAL PRIMARY KEY,
5      category VARCHAR(50),
6      amount NUMERIC
7  );
8
9  -- Remote node (Node_B simulated)
10 CREATE TABLE expense_remote (
11     id SERIAL PRIMARY KEY,
12     category VARCHAR(50),
13     amount NUMERIC
14 );
15

```

Data Output

Messages

Notifications

CREATE TABLE

Query returned successfully in 234 msec.

Query

Query History

Scratch Pad X

```

1  --2: Successful 2PC Transaction
2  -- Start a new transaction
3  BEGIN;
4
5  -- Local insert (Node_A)
6  INSERT INTO expense_local (category, amount)
7  VALUES ('Transport', 1200);
8
9  -- Remote insert (Node_B simulated)
10 INSERT INTO expense_remote (category, amount)
11 VALUES ('Transport', 1200);
12
13 -- Prepare the transaction (Phase 1)
14 PREPARE TRANSACTION 'txn_success';
15
16

```

Data Output

Messages

Notifications

PREPARE TRANSACTION

Query returned successfully in 478 msec.

Total rows:	Query complete 00:00:00.483	CRLF	Ln 15, Col 1
-------------	-----------------------------	------	--------------

Query

Query History

Scratch Pad X

```

1  --verify it's pending:
2  SELECT gid, prepared, owner, database
3  FROM pg_prepared_xacts;
4
5

```

Data Output

Messages

Notifications

	gid	prepared	owner	database
	text	timestamp with time zone	name	name
1	txn_success	2025-10-28 21:37:37.151332+02	postgres	CCR

Query

Query History

Scratch Pad X

```

1  --Check it:
2  SELECT gid, prepared, owner, database FROM pg_prepared_xacts;
3

```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

Showing rows: 1 to 1 Page No: 1 of 1

	gid	prepared	owner	database
	text	timestamp with time zone	name	name
1	txn_fail	2025-10-28 21:43:22.062324+02	postgres	CCR

Query

Query History

Scratch Pad X

```

1  --recover it:
2  COMMIT PREPARED 'txn_fail';
3

```

Data Output

Messages

Notifications

COMMIT PREPARED

Query returned successfully in 223 msec.

Query

Query History

Scratch Pad X

```

1  --see if it is cleared
2  SELECT gid FROM pg_prepared_xacts;
3

```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

gid
text

Query
Query History

Scratch Pad X

```

1  --4: Verification
2  SELECT * FROM expense_local;
3  SELECT * FROM expense_remote;
4

```

Data Output
Messages
Notifications

Showing rows: 1 to 2
Page No: 1 of 1

	id [PK] integer	category character varying (50)	amount numeric
1	3	Transport	1200
2	4	Equipment	2000

A5: Distributed Lock Conflict & Diagnosis (no extra rows)

Query
Query History

Scratch Pad X

```

1  --1. Set up sample table and data
2  CREATE TABLE project (
3      id SERIAL PRIMARY KEY,
4      project_name TEXT,
5      budget NUMERIC
6  );
7
8  INSERT INTO project (project_name, budget)
9  VALUES
10 ('Alpha', 1000),
11 ('Beta', 2000),

```

Data Output
Messages
Notifications

INSERT 0 3

Query returned successfully in 258 msec.

Query	Query History	Scratch Pad X
1	--2 - Session 1 (Node_A)	
2	BEGIN;	
3	UPDATE project SET budget = 1500 WHERE id = 1;	
4	-- Do NOT COMMIT or ROLLBACK yet. Keep this transaction open.	
5		
6		
7		

Data Output	Messages	Notifications
UPDATE 1		
Query returned successfully in 171 msec.		

Query	Query History	Scratch Pad X
1	--3 - Session 2 (Node_B)	
2	BEGIN;	
3	UPDATE project SET budget = 1800 WHERE id = 1;	
4	-- This will hang (waiting for Session 1 to release the lock)	
5		

Data Output	Messages	Notifications
ERROR: current transaction is aborted, commands ignored until end of transaction block		
SQL state: 25P02		

Query	Query History	Scratch Pad X
1	--5- Release the lock (Session 1)	
2	COMMIT;	
3		

Data Output	Messages	Notifications
ROLLBACK		
Query returned successfully in 175 msec.		

QueryQuery History↗

Scratch Pad ×↗

1--6. verify

2SELECT * FROM project WHERE id = 1;

3

4

Data OutputMessagesNotifications↗

⌵⌵⌵⌵⌵⌵⌵⌵SQL

Showing rows: 1 to 1✎Page No: 1of 1⏪⏩⏴⏵

	id [PK] integer✎	project_name text✎	budget numeric✎
1	1	Alpha	1000